

# Composition of Arithmetic Types

## Contents

- [Introduction](#)
- [Goals](#)
  - [Efficiency](#)
  - [Correctness](#)
  - [Usability](#)
- [Definitions](#)
- [Prior Art](#)
  - [Compositional](#)
  - [Non-Compositional](#)
- [Componentization](#)
  - [Example A: Safety](#)
  - [Example B: Elasticity](#)
  - [Example C: Scale](#)
- [Composition](#)
  - [Example A: Elastic Fixed-Point](#)
  - [Example B: Safe Elastic Integer](#)
  - [Example C: Safe Elastic Fixed-point](#)
- [Advanced Topics](#)
  - [Leaky Abstractions](#)
  - [Generic Numeric Functions](#)
  - [Rounding](#)
  - [The 64-bit Limit](#)
  - [Operator Overloads](#)
  - [Integral Constants](#)
  - [User-defined Literals](#)
- [Acknowledgements](#)
- [References](#)

## Introduction

One of the goals of the Numerics TS is to add general-purpose arithmetic types to the library [P0101]. These types are to provide much needed safety and correctness features which are absent from fundamental types.

This paper puts the case for expressing those features as individual components which can then be composed into types with the desired feature set. This approach is referred to as the *compositional* approach and is characterized by class templates with *type* parameters:

```
// approximate a real number by scaling an int32_t down by 16 bits
using f = fixed_point<int32_t, -16>;
```

The compositional approach is contrasted with the *comprehensive* approach which aims to achieve similar goals using class templates with *non-type* parameters:

```
// approximate a real number with 15 integer and 16 fractional digits
using f = fixed_point<15, 16>;
```

## Goals

The design space of arithmetic types aims to satisfy goals which fall into three competing categories: *efficiency*, *correctness* and *usability*.

### Efficiency

One aim of a library-based solution should be to maintain the efficiency of fundamental arithmetic types. Some run-time costs are inevitable additions to existing computation. All other run-time costs should be avoided in pursuit of zero-cost abstraction.

Further, a new wave of specialized hardware is reaching maturity in the form of heterogeneous processing units. These include the GPUs which provide the majority of computing power on modern PCs and mobile devices. The design and capabilities of these units is continually changing in ways which are difficult to predict. An interface which can easily be employed to harness these capabilities is highly valuable. And this may improve on the performance of existing fundamental types.

Over the past few decades, floating-point performance has largely caught up with integer performance. However, this has come at the cost of increased energy consumption. As performance per watt becomes the limiting factor for cloud and mobile platforms, so the economic impetus to use integers to approximate real numbers increases.

In summary, three efficiency-driven goals are:

- zero-cost abstraction over existing fundamental types;
- extensibility and
- increased applicability of integer hardware units.

## Correctness

Native numeric types have a variety of characteristics that can make them difficult to use correctly. Among those within the scope of this document are:

Integers:

- operator overflow from: cast to float, shift left, multiplication, addition;
- conversion overflow from: cast to narrower types and unsigned types;
- undefined signed overflow vs modulo unsigned overflow;
- surprising promotion rules;
- divide-by-zero;
- underflow/flushing;
- variation across architectures;
- endianness;
- most negative value;
- inconsistent/inaccurate rounding;
- lack of fractional digits.

Floating-point:

- precision varied by scale;
- silent transition to special values;
- special values causing loss of strict weak ordering;
- lack of associativity and commutativity;
- non-determinism;
- divide-by-zero.

One reason the list is shorter for floating-point is because of extra work it performs at run-time. But opportunities exist to address many of these items without run-time cost: increasingly, smarter numeric types can shift cost to the compiler. However, one problem is that the range of solutions is wide with no single solution to satisfy all use cases.

## Usability

There are two overwhelming reasons why any new arithmetic types should emulate existing fundamental ones:

1. They require little more than a rudimentary grasp of elementary algebra.
2. They are a language feature with which almost all users are familiar.

It is true that many users complain that arithmetic types are hard to use. What they often mean is that they are hard to use *correctly*: the learning curve is mostly smooth but hits a bump whenever a [Correctness](#) issue is encountered.

## Definitions

A class is recognized as an arithmetic type if it possesses certain similarities with fundamental arithmetic types. In particular, it must have value semantics and arithmetic operators. It is often composed of simpler types such as fundamental arithmetic types. However, it is only said to have a compositional interface if the choice of type is a feature of its interface. Thus

compositional interfaces tend to be part of class templates.

For example,

```
template<class Rep>
class A {
    Rep data;
};

template<int Bits, bool Signed>
class B;

template<>
class B<32, false> {
    uint32_t data;
};

A<uint32_t> a;
B<32, false> b;
```

both `a` and `b` are composed of the same data type but only `a` is said to have a compositional interface.

An important property of compositional interfaces are that they can be nested:

```
template<class T>
class C;

C<A<T>> c;
```

## Prior Art

### Compositional

Examples of existing types with a compositional interface include:

- integer types which test for overflow at run-time [[Chromium](#), [P0228](#), [type\\_safe](#)];
- fixed-point real number approximations [[P0037](#), [fp](#)];
- arbitrarily wide integers [[Boost.Multiprecision](#)];
- auto-widening integers [[elastic integer](#)];
- complex numbers (`std::complex`) and
- statically scaled time interval (`std::chrono::duration`).

### Non-Compositional

Counter-examples include Lawrence Cowl's fixed-point paper [[P0106](#)] which proposes a set of *comprehensive* class templates. These produce types which solve all or most of the integer correctness issues listed in section, [Correctness](#).

Other counter-examples include:

- arbitrarily wide integers [[P0104](#), [wide\\_int](#), [Boost.Multiprecision](#)] and
- an integer type which detects overflow conditions [[bounded::integer](#)].

(Note that [[Boost.Multiprecision](#)] earns a place on both lists as it uses both compositional and non-compositional techniques.)

## Componentization

This section introduces several examples of arithmetic components and explores how they behave when specialized using fundamental types. Each is compared to equivalent comprehensive classes and functionality from [[P0106](#)].

### Example A: Safety

There are many examples of libraries which aim to add run-time error detection to integers. These are useful for addressing:

- uninitialized data;
- divide by zero;
- overflow due to arithmetic operations;
- overflow due to cast from wider integer;
- overflow due to cast from floating-point type and
- most negative value.

The `integral` (signed) and `cardinal` (unsigned) class templates from [P0106] perform such a role using non-type template parameters, e.g.

```
template<int Crng, overflow Covf = overflow::exception>
class integral;
```

where `Crng` specifies the range of the value in bits and `overflow` is an enum which specifies what happens if overflow is detected. The values of `overflow` include:

- `undefined` - much like signed integer overflow behavior;
- `abort` - forces program termination;
- `exception` - throws an exception;
- `saturate` - limits the value to the maximum/minimum allowed by the range.

The complete list of values is found in [P0105] and covers a wide range of use cases. However, it is necessarily finite. If any new mode is desired, it must wait on a revision to the standard.

The compositional equivalent might look like

```
template<class Rep = int, class ErrorHandler = throwing_handler<Rep>>
class safe_integer {
    Rep rep_;
public:
    // ...
};
```

where `ErrorHandler<Rep>` specifies what happens under various overflow conditions:

```
template<class Rep>
struct throwing_handler {
    static constexpr Rep const& max_overflow(Rep const& r) noexcept {
        throw std::overflow_error("positive overflow");
    }
    // ...
};
```

Alternatives to `throwing_handler` will match the values of `enum, overflow`. For example, `saturate_handler` might correspond to `overflow::saturate`. However, the user is free to implement alternative handlers as they choose.

In practice, all of these integer types: `safe_integer`, `integral` and `cardinal` behave similarly to fundamental integers *until* overflow occurs:

```
safe_integer<uint8_t> n = 255; // interface hints that n will resemble uint8_t
++ n; // but when its range is exceeded, an exception is thrown
```

```
safe_integer<int, saturate_handler<int>> i = 1e100; // i == INT_MAX
```

Crucially, `safe_integer<T>` gives the user a strong hint that they can expect similar characteristics to `T`. If the user is familiar with `int` and chooses it to specialize `safe_integer`, they should have a good idea of how it will behave.

But the most remarkable safety feature of [P0106]'s types is not supported by `safe_integer` at all.

## Example B: Elasticity

Elasticity is the ability to expand range in order to contain a growing value. For instance, when two 8-bit numbers are summed, the result cannot exceed 9 bits. And when two 20-bit numbers are multiplied, the result cannot exceed 40 bits. Thus, an elastic type tracks the exact number of bits used and returns suitably widened results from binary arithmetic operations. This eliminates a common class of overflow errors without the need for run-time checks.

[P0106]'s integer types achieve elasticity by adjusting range:

```
integer<4> n = 15; // 4-digit signed integer
auto nn = n * n; // 8-digit signed integer
```

Consider an equivalent compositional type, [elastic integer]:

```
template<int Digits, class Narrowest = int>
class elastic_integer;
```

`Digits` is equivalent to `Crng` - the number of digits of capacity.

`Narrowest` serves a similar purpose to the `Rep` parameter of `safe_integer`. However, it is not guaranteed to be the type of

`elastic_integer`'s member variable. For example,

```
elastic_integer<10, int16_t> n;
```

uses an `int16_t` to store a value in the range,  $[1024..1024]$ . To avoid overflow, `elastic_integer`'s binary arithmetic operators 'stretch' the value of `Digits`:

```
auto nn = n * n; // elastic_integer<20, int16_t>
```

As `int16_t` cannot store a 20-bit value, a wider type is used. `Narrowest` might seem superfluous - given `Digits` can override the type's width. However, it serves two important purposes.

Firstly, storage requirements vary depending on use case. When serializing to file, compactness may be preferable, in which case, `int8_t` or `uint8_t` is appropriate. In most other situations, performance is paramount and `int` makes a better choice. It is hoped that `Narrowest` makes the most sense to users who are already familiar with this trade-off.

Secondly, `Narrowest` determines the set of types from which a member variable will be chosen. If `int8_t` is specified, then `int16_t`, `int32_t` and `int64_t` will be used as width dictates whereas `uint8_t` will expand to `uint16_t`, `uint32_t` and `uint64_t`.

There is nothing to prevent `Narrowest` being a custom arithmetic type of the user's choosing. One stipulation, however, is that `elastic_integer` is able to choose a wider type as required. The details of how this is achieved are covered in [\[P0381\]](#).

## Example C: Scale

Fixed-point numbers have always held certain efficiency and correctness advantages over floating-point types. Modern language features mean that they are also much simpler to work with.

[\[P0106\]](#)'s integral and cardinal class templates are complemented by fixed-point templates, `negatable` and `nonnegative` respectively, e.g.:

```
template<int Crng, int Crsl, round Crnd = rounding::tie_to_odd, overflow Covf = overflow::exception>
class negatable;
```

They come with two additional template parameters. `Crsl` specifies resolution. For example,

```
nonnegative<8, -4> n;
```

instantiates a variable with 8 integer bits and 4 fractional bits. Its maximum, minimum and lowest values are 255.9375, 0.0625 and 0 respectively.

`Crnd` specifies the rounding mode as described in [\[P0105\]](#). The default, `rounding::tie_to_odd`, represents the best information preservation. The list of alternatives supplied by `rounding` is extensive but - just as with `overflow` - it is finite.

An alternative, compositional fixed-point solution is proposed in [\[P0037\]](#):

```
template<typename Rep = int, int Exponent = 0>
class fixed_point;
```

`Exponent` governs scaling in much the same way as the exponent of an IEEE 754 floating-point type. It is equivalent to `Crsl`.

Aside from its compositional interface the most obvious difference is that `fixed_point` does not handle rounding. Rounding and scaling are orthogonal concerns. It should be possible to devise a `rounding_integer` component which handles rounding separately. See [Rounding](#) for further discussion.

There are a wide variety of strategies for performing fixed-point arithmetic. However, `fixed_point` is in a category which - broadly-speaking - follows the lead of [\[P0106\]](#) and tracks the result of `Rep`'s arithmetic - rather than imposing a different result.

For instance, when two `fixed_point` values are multiplied together, all that happens is that their internal values are multiplied and the result is used to create a new `fixed_point` object of the appropriate type. No scaling or conversion occurs, resulting in the highest chance of zero-cost abstraction.

To illustrate, the operator overload looks something like

```
template<class RepA, int ExponentA, class RepB, int ExponentB>
auto operator*(fixed_point<RepA, ExponentA> const & a, fixed_point<RepB, ExponentB> const & b) {
    // deduce the type of the raw result
    using result_rep = decltype(b.data() * a.data());

    // calculate the resultant exponent
    constexpr auto result_exponent = ExponentA + ExponentB;

    // multiply the raw values and store the result in a new object of a new type
```

```

    return fixed_point<result_rep, result_exponent>::from_data(b.data() * a.data());
}

```

where `data()` and `from_data()` access `fixed_point`'s private member variable. (See section, "[Leaky Abstractions](#)", for a discussion of this aspect of interface design.)

The aim of `fixed_point` operators is to avoid adding any complexity to underlying arithmetic operations. For example, these two samples are equivalent:

```

// sample 1 - bare-metal fixed-point arithmetic
auto a = (int8_t)(7.f * 8);      // the value 7 stored in a byte with 3 fractional bits
auto b = (int8_t)(3.125f * 16);  // the value 3.125 stored in a byte with 4 fractional bits
auto c = a * b;                 // the value 21.875 stored in an `int` with 7 fractional bits
auto d = (float)c / 128;        // 21.875f

// sample 2 - type-safe fixed-point arithmetic
auto a = fixed_point<int8_t, -3>(7.f);    // the value 7 stored in a byte with 3 fractional bits
auto b = fixed_point<int8_t, -4>(3.125f);  // the value 3.125 stored in a byte with 4 fractional bits
auto c = a * b;                          // the value 21.875 stored in an `int` with 7 fractional bits
auto d = (float)c;                       // 21.875f

```

Here we see that promotion rules help avoid errors and maximize hardware resources. In this case, the fundamental integral type system shines and the results are efficient, correct and usable. Unfortunately, a fair degree of good fortune is involved. Without safety or elasticity, `fixed_point<int, E>` types have a tendency to overflow.

## Composition

In the previous section, three possible arithmetic components are introduced. Each component can be instantiated using fundamental integer types such as `int`. The results are incrementally enhancements to `int`. This section illustrates ways in which nested composition can combine these enhancements.

### Example A: Elastic Fixed-Point

Fixed-point arithmetic is especially susceptible to overflow. Integers typically represent small quantities whereas fixed-point values often saturate their range:

```

int32_t toes = 10; // uses 4 bits
fixed_point<int32_t, -30> probability = 0.9; // uses 30 bits

```

This means that hand-rolled fixed-point arithmetic involves keeping track of - not only scale but also - range:

```

// square a floating-point value with 16 bits of fractional precision
float square(float input) {
    auto fixed = static_cast<int32_t>{input * 65536.f};
    auto prod = int64_t{fixed} * fixed; // result must be widened
    return prod / 4294967296.f; // gotcha: scale has changed
}

```

This error-prone work can be automated with a template composed of `fixed_point` and `elastic_integer`:

```

template<int IntegerDigits, int FractionalDigits, class Narrowest = int>
using elastic_fixed_point = fixed_point<elastic_integer<IntegerDigits+FractionalDigits, Narrowest>, -FractionalDigits>;

```

The same `square` function is now clearer:

```

float square(float input) {
    auto fixed = elastic_fixed_point<15, 16>{input};
    auto prod = fixed * fixed;
    return static_cast<float>(prod);
}

```

In both cases, a modern optimizing compiler produces the same x86-64 assembler instructions [[godbolt](#)], e.g.:

```

square(float):
    mulss    xmm0, DWORD PTR .LC0[rip]
    cvttss2si    eax, xmm0
    pxor     xmm0, xmm0
    cdqe
    imul     rax, rax
    cvtsi2ssq    xmm0, rax
    mulss    xmm0, DWORD PTR .LC1[rip]
    ret
.LC0:
    .long    1199570944

```

```
.LC1:
    .long    796917760
```

Multiplication is the easiest arithmetic operator to deal with. Addition and subtraction require that both operands be scaled to the same amount before addition. But toughest of all is division. Consider:

```
fixed_point<int8_t, -6> dividend = 0.5, divisor = 1.0;
auto quotient = dividend / divisor;
```

Under the hood, `dividend` and `divisor` are represented by values 32 and 64 respectively. Whenever the divisor is greater than that dividend, integer division results is 0.

The problem is that integer and floating-point types deal with division in a different way. Integers produce a remainder, whereas floating-point types have fractional values. Despite being represented by integer arithmetic, fixed-point arithmetic favors the fractional approach.

To implement fractional division in a practical way (and excepting inevitable precision loss that comes from digital representation of rational numbers), `fixed_point` automatically widens and shifts the dividend prior to division.

In the above example, the integer arithmetic occurs as follows:

```
int8_t dividend = 0.5 * 64;    // 32 ; fixed_point<int8_t, -6>{.5}
int8_t divisor = 1.0 * 64;    // 64 ; fixed_point<int8_t, -6>{1.}

// result adds number of divisor's integer digits to number of dividend's fractional digits,
// producing fixed_point<int16_t, -7>
auto intermediate = (decltype(dividend*divisor){dividend} << 7)    // 4096 ; fixed_point<int, -13>{.5}
auto q = intermediate / divisor; // 64 ; fixed_point<int, -7>{.5}
```

The `intermediate` variable is added here to illustrate how the dividend is shifted ahead of integer division in order to ensure correct scale and resolution. The result is an `int` - the same type as a multiplication - because both fixed-point multiplication and division results require the same number of digits. While this is twice as wide as necessary in the case of fundamental types, `elastic_integer` tracks bits more accurately and no unnecessary width is added.

So using `elastic_fixed_point`, quotient becomes `elastic<7, 7>{.5}`.

## Example B: Safe Elastic Integer

With the correct set of behaviors it's possible to instantiate an integer type which has excellent correctness and usability characteristics. The following composite is practically overflow-proof and performs minimal run-time checks:

```
template<int IntegerDigits, class Rep = int, class ErrorHandler = throwing_handler<Rep>>
using safe_elastic_integer = safe_integer<elastic_integer<IntegerDigits, Rep>, class ErrorHandler>;
```

The order of the nesting is crucial in ensuring that checks are minimized. This is because there is overlap in the concerns of the two components: `safe_integer` traps binary arithmetic overflow whereas `elastic_integer` avoids it. Detection of something that isn't going to happen is a needless cost. By being aware of the types involved in operations, `safe_integer` can be more selective about checks.

Consider `safe_integer`'s multiplication operator:

```
template<class RepA, class RepB, class ErrorHandler>
auto operator*(safe_integer<RepA, ErrorHandler> const & a, safe_integer<RepB, ErrorHandler> const & b) {
    using result_type = decltype(a.data() * b.data());
    constexpr auto digits_a = std::numeric_limits<RepA>::digits;
    constexpr auto digits_b = std::numeric_limits<RepB>::digits;
    constexpr auto digits_result = std::numeric_limits<result_type>::digits;
    if constexpr (digits_a + digits_b >= digits_result) {
        // perform overflow test
    }
    else {
        // skip overflow test
    }
    return a.data() * b.data();
}
```

We see that unnecessary run-time checks are eliminated.

This holds - not only for `elastic_integer` but also - for fundamental types that are implicitly promoted:

```
safe_integer<int8_t> a, b;
auto c = a * b; // c is safe_integer<int>; no need for run-time check
```

## Example C: Safe Elastic Fixed-point

This sub-section passes over the third and final pairing of components, `safe_fixed_point`, and instead considers the composition of all three components:

```
template<
    int IntegerDigits,
    int FractionalDigits = 0,
    class Narrowest = int,
    class ErrorHandler = throwing_handler<Narrowest>>
using safe_elastic_fixed_point =
    fixed_point<
        safe_integer<
            elastic_integer<
                IntegerDigits+FractionalDigits,
                Narrowest>,
                ErrorHandler>,
            -FractionalDigits>;
```

This type is a run-time-efficient, overflow-proof real number approximation:

```
safe_elastic_fixed_point<5, 0> a = 30;
safe_elastic_fixed_point<2, 0> b = 2;
auto c = a * b; // safe_elastic_fixed_point<7, 0>{60}
auto d = a + b; // safe_elastic_fixed_point<6, 0>{32}
auto e = c / d; // safe_elastic_fixed_point<7, 6>{1.875}
a -= 10; // safe_elastic_fixed_point<5>{20}
++ b; // safe_elastic_fixed_point<2>{3}
++ b; // exception!
```

The above is an abstraction over the following code

```
int a = limit<overflow::exception>(-32, 31, 30);
int b = limit<overflow::exception>(-4, 3, 2);
int c = a * b;
int d = a + b;
int e = (c * (1 << 6)) / d;
a = limit<overflow::exception>(-32, 31, a - 10);
b = limit<overflow::exception>(-4, 3, b + 1);
b = limit<overflow::exception>(-4, 3, b + 1);
```

where `limit` is a function template which handles overflow at run-time. (A similar facility is detailed in [\[P0105\]](#).)

## Advanced Topics

It is not claimed that a complete set of solutions has been presented so far. However, it is hoped that much of the appeal of the compositional approach is apparent. This section is a grab bag of topics of relevance to the types of a numerics TS. These topics are discussed with the compositional approach in mind.

### Leaky Abstractions

As mentioned in section, [Scale](#), `fixed_point` has member functions for accessing its data member:

```
template<class Rep, int Exponent>
Rep const & fixed_point<Rep, Exponent>::data() const;

template<class Rep, int Exponent>
static Rep const & fixed_point<Rep, Exponent>::from_data(Rep const &) const;
```

These are analogous to the `data()` member function and allocator of `vector`. They are provided for situations when the user wishes to lift the hood and access the engine directly. For example, a certain serialization or compression strategy may be easier to implement with raw integers.

It's important to note that these accessors are named functions. They don't exist for fundamental types and the user should know that they are intentionally violating the abstraction whenever they use them.

### Generic Numeric Functions

An example of when accessing the underlying representation may be necessary is with the numeric support functions described in [P0105](#). For instance, consider the conversion function prototype:

```
template<rounding mode, typename T, typename U>
T convert(U value);
```



It is expected that overloads of `convert` will perform rounding on fundamental floating point types.

However, applying such functions to composite types might also be desirable. For this reason a generic overload is worth considering:

```
template<rounding mode, typename T, typename U> T convert(U const & value) {
    return T::from_data(convert(value.data()));
}
```

Such functions can form chains which 'mine' into a composite type for its fundamental type and perform the desired operation on it. This is only possible where all components have a common set of accessors.

## Rounding

[P0106]'s `negatable` and `nonnegative` offer a choice of rounding modes whereas their integer equivalents do not. However, rounding is orthogonal to real number approximation and might best be embodied in its own component, `rounding_integer`.

However, it is yet to be determined whether such a component is the right approach. It may be a more practical solution to roll rounding functionality into `safe_integer`.

## The 64-bit Limit

Another component which has been brushed over so far is `wide_integer`. Multiplication of `elastic_integer` results in double-wide types. This quickly hits limits:

```
elastic_integer<> n; // typically 31 digits
auto n2 = n * n;    // 62 digits
auto n4 = n2 * n2;  // 124 digits
```

There is no standard facility to represent values greater than 64 bits with fundamental types. This limit routinely causes expressions to fail compilation. For this reason, either a `wide_integer` is needed or `elastic_integer` must handle multi-word values. Papers, [P0104] and [wide\_int] both explore this topic.

## Operator Overloads

This paper is also light on details regarding operator overloads. However, they play a vital role in providing the semantics of arithmetic types. It is a non-trivial undertaking to support a complete set of operators for a single interface yet the compositional approach requires multiple interfaces. Thus the number of operator overloads (and special functions) is a burden. Any language features which can reduce this boilerplate are welcome.

Many functions are lightweight wrappers over a `Rep` type. An example of a feature which could reduce such boilerplate is default comparison [N4475].

## Integral Constants

As shown in sub-section, [The 64-bit Limit](#), `elastic_integer` be excessively 'stretched' by literal type, `int`:

```
elastic_integer<2> x = 3; // x uses 2 digits
auto y = x * 2; // suddenly result uses over 30 digits to store value 6
```

If the value (and therefore the range) is known at compile time, a much narrower result type can be generated:

```
auto z = x * std::integral_constant<int, 2>; // only 4 digits used in result
```

Binary bit shift operations are a major benefactor if `integral_constant` operands. In this example

```
auto a = fixed_point<int, 0>{1};
auto b = a << std::integral_constant<int, 10>; // fixed_point<int, 10>{1024}
```

both `a` and `b` store the same underlying value. No actual shift is required.

Another technique which uses compile-time constants to avoid run-time computation involves division. Any division can be approximated to varying degrees by different combinations of multiplication and bit shift. For example, to divide an `int16_t` by 7 using an `int32_t`:

```
int16_t divide_by_seven(int16_t n) {
    return (n * 37449) >> 18;
}
```

Unfortunately, the coefficient is costly to calculate and the operation requires additional width. If the divisor and headroom is

known at compile-time, the calculation can potentially outperform a division operation.

## User-defined Literals

It is possible to improve on `std::integral_constant` in various ways. Consider:

```
template<
    class Integral,
    Integral Value,
    int Digits = calculate_digits(Value),
    int Zeros = calculate_zeros(Value)>
class const_integer {};
```

This type resembles `integral_constant` with two additional non-type parameters:

- Digits - the numbers of digits required to store `Value`;
- Zeros - the number of trailing binary 0s between the least significant binary 1 and the radix position.

For example:

```
auto m = const_integer<int, 10>{}; // Value = 0b1010, Digits = 4, Zeros = 1
auto n = const_integer<int, 96>{}; //xx Value = 0b1100000, Digits = 7, Zeros = 5
```

Now consider a user-defined literal which provides a shorthand for `const_integer` values:

```
auto m = 10_c; // const_integer<int, 10>{}
auto n = 96_c; // const_integer<int, 96>{}
```

Finally, combine these user-defined literals with class template deduction. The `Digits` and `Zeros` parameters of `const_integer` can be matched with the template parameters of composite types:

```
auto e = elastic_integer(10_c); // elastic_integer<4, int>{10}
auto f = fixed_point(96_c); // fixed_point<int, 5>{96}
```

The result is easy to read formulation of tailored composite types. Unfortunately, class template deduction does not extend to aliases. Because of this, it is not yet possible to write definitions such as:

```
auto kibi = elastic_fixed_point(1024_c); // stores value 1024 using 2 bits
auto half = kibi / 2048_c; // stores value 0.5 using 3 bits
```

## Acknowledgements

Thanks to Lawrence Crowl and Jens Maurer for valuable feedback on early drafts. Special thanks to Michael Wong.

## References

[Boost.Multiprecision](#), `cpp_int`, John Maddock and Christopher Kormanyos  
[bounded::integer](#), `bounded::integer`, David Stone  
[elastic\\_integer](#), `elastic_integer`, John McFarlane  
[fp](#), `fp` fixed-point library, Matheus Izvekov  
[godbolt](#), `elastic_fixed_point` example on CompilerExplorer, John McFarlane  
[Chromium](#), safe arithmetic excerpts from Chromium project, Justin Schuh  
[N4475](#), Default comparisons, Bjarne Stroustrup  
[P0037](#), Fixed-Point Real Numbers, John McFarlane  
[P0101](#), An Outline of a C++ Numbers Technical Specification, Lawrence Crowl  
[P0104](#), Multi-Word Integer Operations and Types, Lawrence Crowl  
[P0105](#), Rounding and Overflow in C++, Lawrence Crowl  
[P0106](#), C++ Binary Fixed-Point Arithmetic, Lawrence Crowl  
[P0228](#), A Proposal to Add Safe Integer Types to the Standard Library Technical Report, Robert Ramey  
[P0381](#), Numeric Width, John McFarlane  
[type\\_safe](#), `type_safe`, Jonathan Muller  
[wide\\_int](#), A Proposal to add `wide_int` Template Class, Igor Klevanets, Antony Polukhin